

Intégration Continue (CI/CD) et Script de Pipeline

Vous allez simuler un **Runner CI/CD** (l'agent automatisé de GitLab CI, GitHub Actions ou Azure DevOps).

L'orchestrateur ne prend pas en compte de la couleur du texte. Il n'a besoin que de deux choses :

1. Un rapport structuré au format **XML (JUnit)** pour afficher des graphiques sur le portail web.
2. Un **Code de sortie (Exit Code)** strict (0 = Succès, 1 = Échec) pour savoir s'il doit continuer le déploiement ou l'annuler.

Vous allez créer le script Run-Pipeline.ps1 qui servira de point d'entrée exclusif à l'automate.

Exercice 1 : Création du Script "Wrapper" (Lanceur)

1. Dans votre dossier racine C:\Projets\InfraTools\, créez un nouveau fichier nommé Run-Pipeline.ps1.
(Ce fichier ne contient pas de tests, il sert uniquement à Lancer les tests).
2. Insérez un bloc try/catch global. Ce bloc est vital car si une erreur inattendue se produit (ex: le dossier des tests a été supprimé), l'orchestrateur doit être prévenu par un code 1.

Squelette de syntaxe :

```
Write-Host "=== DÉMARRAGE DU JOB CI/CD : TESTS UNITAIRES ===" -ForegroundColor Cyan
```

```
try {  
    # Le code Pester ira ici  
}  
catch {  
    Write-Host "ERREUR CRITIQUE DE LA PIPELINE : $($_.Exception.Message)"  
    -ForegroundColor Red  
    exit 1  
}
```

Exercice 2 : Configuration de Pester pour la Machine (Headless)

À l'intérieur de votre bloc try, vous allez configurer Pester non plus pour un humain, mais pour la CI/CD.

1. Instanciez un objet de configuration Pester :
`$Config = New-PesterConfiguration`
2. Définissez le chemin cible des tests :
`$Config.Run.Path = ".\Tests\"`
3. **Génération du Rapport XML :**
Activez la sortie des résultats et paramétrez le format standardisé NUnit :
`$Config.TestResult.Enabled = $true`
`$Config.TestResult.OutputFormat = "NUnitXml"`
`$Config.TestResult.OutputPath = ".\artifacts\test-results.xml"`
4. **Le couperet de la CI/CD (Exit Code) :**
Demandez formellement à Pester de forcer le processus PowerShell à quitter avec le code 1 si au moins un test échoue (c'est ce qui fait "planter" le Job CI dans GitLab/GitHub) :
`$Config.Run.Exit = $true`
5. Lancez l'exécution avec cette configuration :
`Invoke-Pester -Configuration $Config`

Exercice 3 : La Simulation de l'Agent CI/CD (CMD)

Pour tester ce script, vous ne devez pas le lancer depuis votre éditeur Visual Studio Code (car le exit fermerait votre session !). Vous allez simuler le comportement d'un agent logiciel externe.

1. Créez un dossier artifacts à la racine de votre projet (pour accueillir le fichier XML) :
`New-Item -Path ".\artifacts" -ItemType Directory -Force`
2. Ouvrez une invite de commandes classique DOS (tapez cmd dans le menu Démarrer Windows).
3. Naviguez vers votre projet :
`cd C:\Projets\InfraTools`
4. Demandez à l'interpréteur d'exécuter votre script (exactement comme le ferait un fichier .gitlab-ci.yml) :
`powershell.exe -NoProfile -File .\Run-Pipeline.ps1`
5. **L'épreuve de vérité (Vérification du code retour) :**
Une fois le script terminé, demandez au système Windows quel a été le code de sortie du dernier processus (l'équivalent de ce que lit l'orchestrateur) :
Tapez dans la console CMD : echo %errorlevel%
Si tous vos tests sont passés au TP précédent, le résultat doit être 0.

Exercice 4 : Casser la Pipeline (Le rôle du gardien)

Une pipeline CI/CD n'est utile que si elle bloque les mauvaises configurations. Nous allons volontairement saboter un script pour voir si notre usine logicielle réagit.

1. Ouvrez votre script source Public\Get-InfraDisque.ps1.
2. Introduisez une erreur (par exemple, modifiez une variable ou divisez par zéro), puis sauvegardez.

3. Retournez dans votre invite de commande DOS (cmd).
4. Relancez la pipeline :
`powershell.exe -NoProfile -File .\Run-Pipeline.ps1`
5. Vérifiez à nouveau le code de retour système :
`echo %errorlevel%`
Le résultat doit désormais être 1. L'orchestrateur aurait bloqué le déploiement en production !
6. Ouvrez le fichier généré `artifacts\test-results.xml` avec un navigateur ou un éditeur de texte. Constatez comment les erreurs sont formatées en balises pour être digérées par les interfaces web DevOps.

(N'oubliez pas de corriger votre erreur dans `Get-InfraDisque.ps1` après l'exercice !)